

Automatically Scale EC2 Instances Based on Traffic Load

Introduction

Cloud computing enables organizations to dynamically manage computing resources based on demand. One of the key challenges in cloud infrastructure is handling fluctuating traffic while maintaining availability and minimizing cost. This project focuses on implementing an automated scaling solution using Amazon Web Services, where EC2 instances automatically scale up or down based on real-time traffic load. The system eliminates the need for manual intervention and ensures efficient utilization of resources.

Objective of the Project

The main objective of this project is to design and deploy an AWS-based architecture that automatically adjusts the number of EC2 instances according to traffic demand. The project aims to ensure high availability, fault tolerance, and cost optimization by scaling resources dynamically using predefined policies and monitoring metrics.

Problem Statement

In traditional server-based architectures, handling sudden traffic spikes requires manual provisioning of servers, which is time-consuming and inefficient. Over-provisioning leads to unnecessary costs, while under-provisioning causes performance degradation. This project addresses these issues by implementing an automated scaling mechanism that responds to traffic changes in real time.

Architecture Overview

The architecture consists of an Application Load Balancer that distributes incoming user requests across multiple EC2 instances managed by an Auto Scaling Group. The Auto Scaling Group uses a launch template to create identical EC2 instances. CloudWatch continuously monitors CPU utilization and triggers scaling actions based on predefined thresholds.

Implementation

A launch template was created with Amazon Linux 2 and a user data script to install and configure a web server automatically during instance launch. An Application Load Balancer was configured to route incoming HTTP traffic to healthy EC2 instances. A scaling policy based on CPU utilization was applied to automate scaling actions.

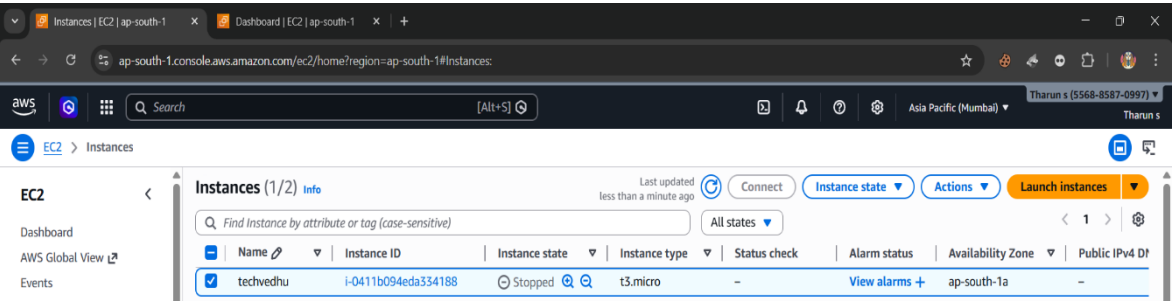
Monitoring and Testing

CloudWatch was used to monitor CPU utilization and instance count metrics. Traffic simulation caused CPU utilization to increase, triggering automatic scale-out. When traffic decreased, scale-in occurred automatically, validating the solution.

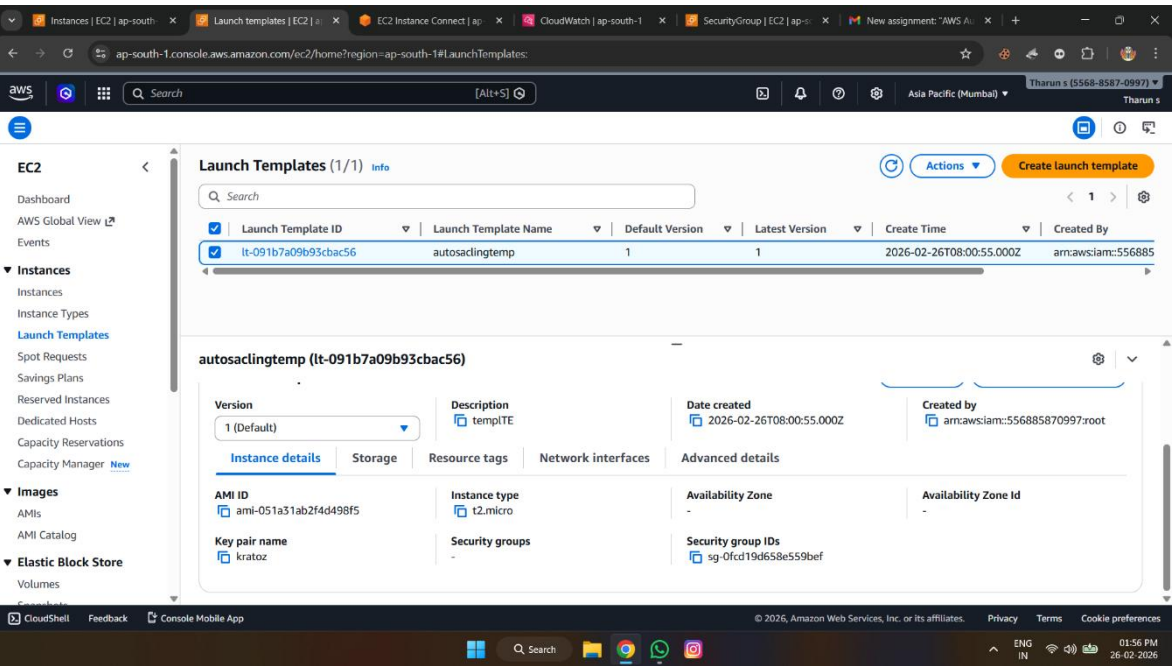
Conclusion

This project successfully demonstrated automatic scaling of EC2 instances based on traffic load. The system improved reliability, scalability, and cost efficiency, making it suitable for real-world cloud applications.

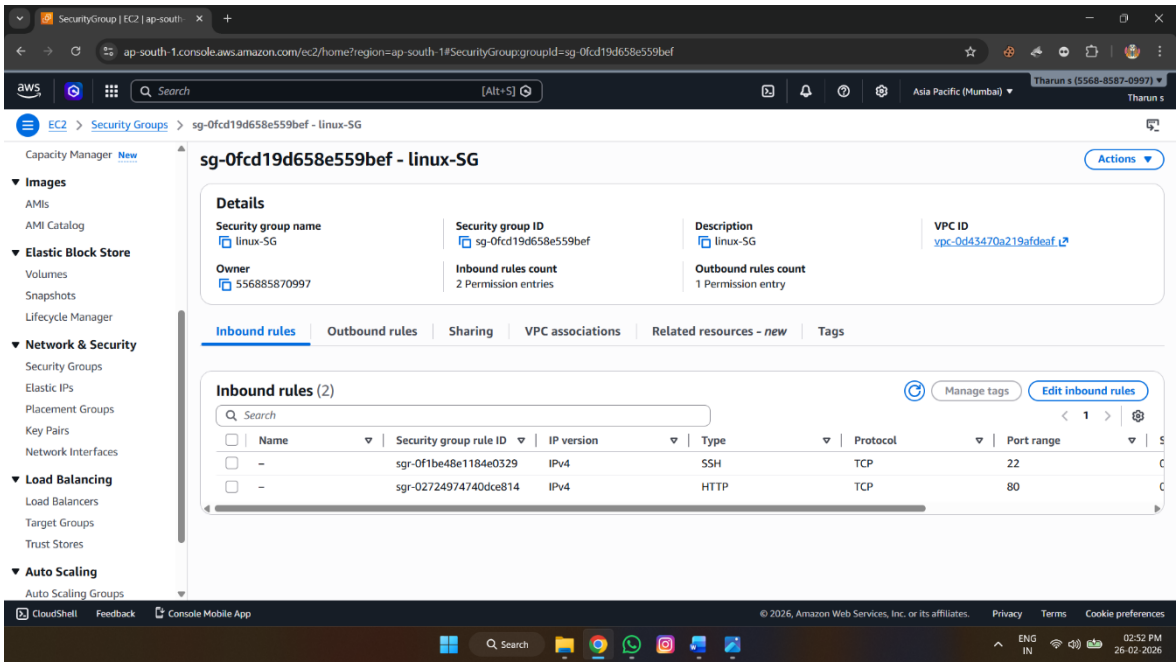
SCREENSHOT



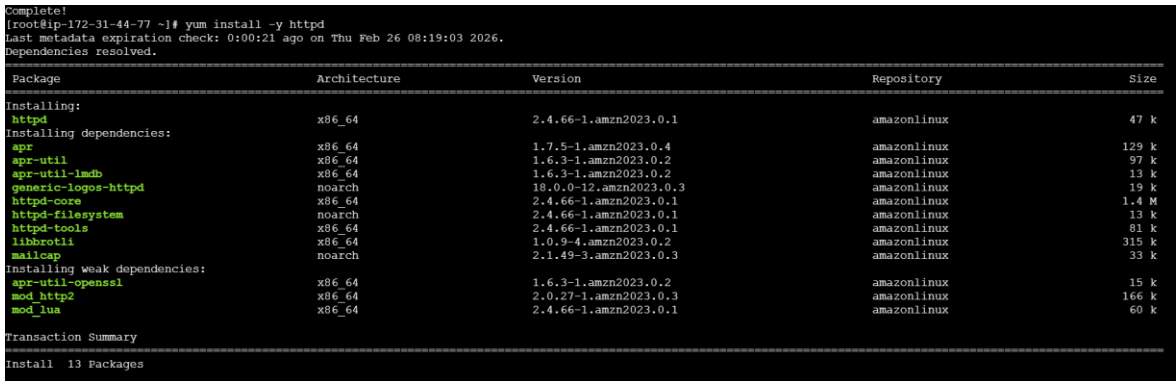
Launch the Ec2 instance



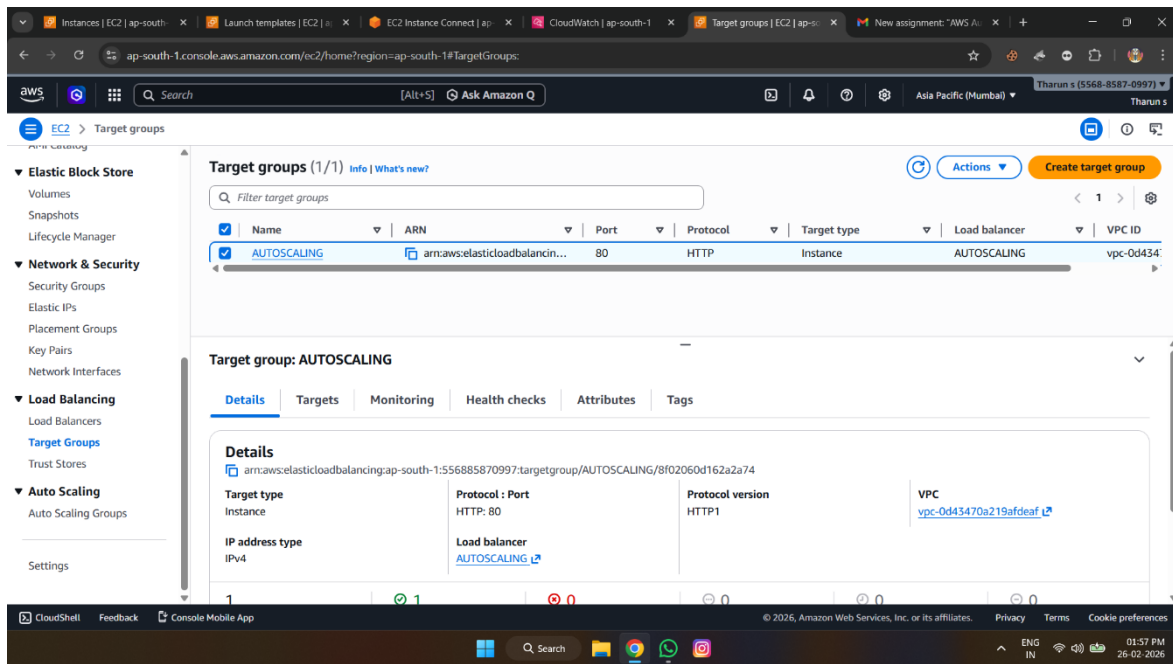
Launch Template



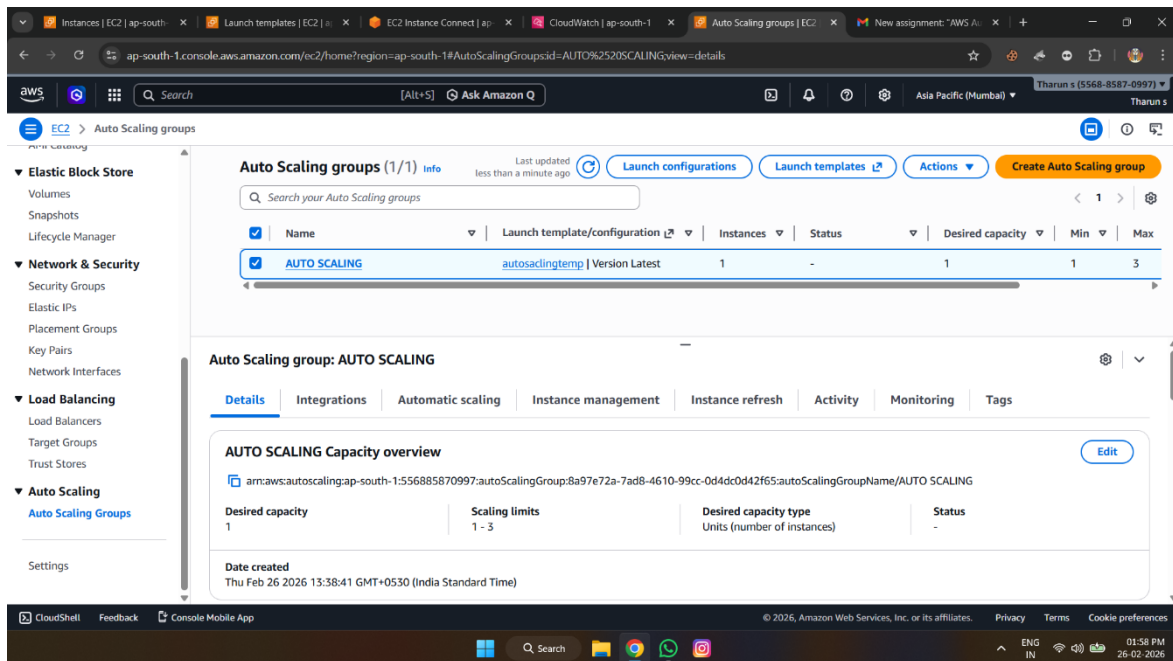
Security Group



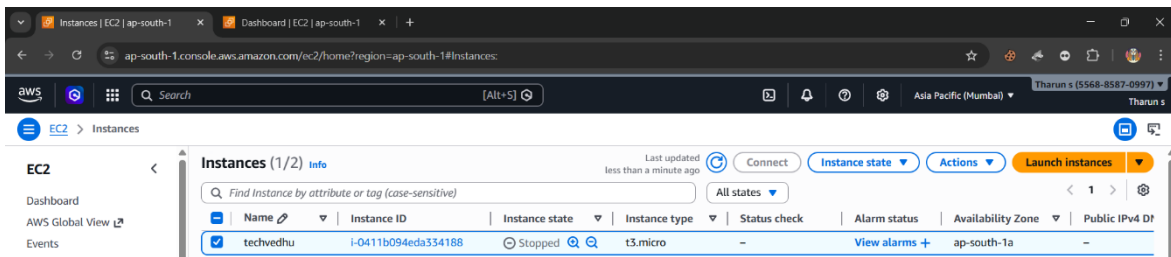
CRTEATION Security Group



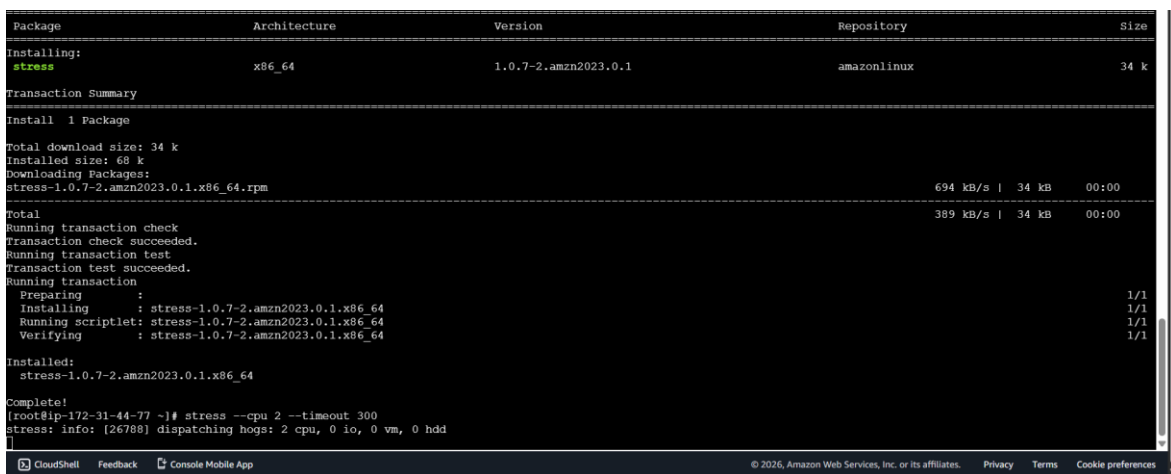
Target Group



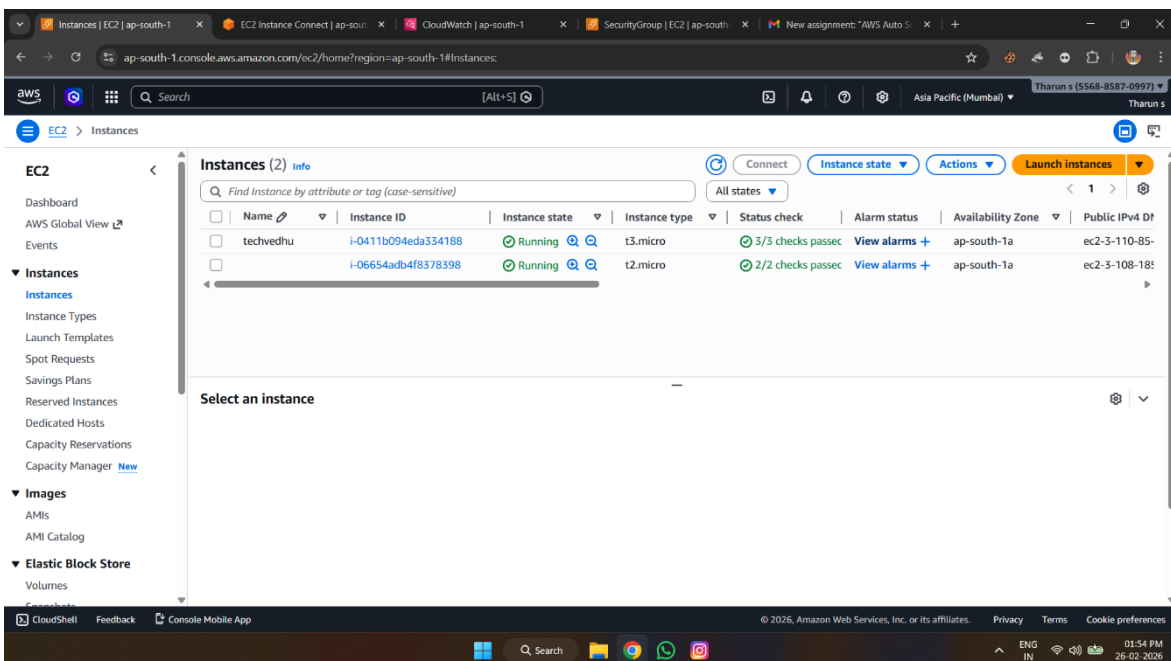
ASG Configuration



Instances Before Load



Traffic Simulation



Instances After Scaling

```
Installed!
stress-1.0.7-2.amzn2023.0.1.x86_64

Complete!
[root@ip-172-31-44-77 ~]# stress --cpu 2 --timeout 300
stress: info: [26788] dispatching hogs: 2 cpu, 0 io, 0 vm, 0 hdd
stress: info: [26788] successful run completed in 300s
[root@ip-172-31-44-77 ~]# stress --cpu 2
stress: info: [27136] dispatching hogs: 2 cpu, 0 io, 0 vm, 0 hdd
```

Load Balancer Output

The screenshot displays the AWS Management Console for the 'ap-south-1' region. The 'Instances' page is active, showing a list of two EC2 instances. The left sidebar contains navigation links for EC2 services like Dashboard, AWS Global View, Events, and various instance types. The main content area features a table with instance details and a 'Select an instance' section below it.

	Name	Instance ID	Instance state	Instance type	Status check	Alarm status	Availability Zone	Public IPv4 D
☐	techvedhu	i-0411b094eda334188	Running	t3.micro	3/3 checks passed	View alarms +	ap-south-1a	ec2-3-110-85-
☐		i-06654adb4f8378398	Running	t2.micro	2/2 checks passed	View alarms +	ap-south-1a	ec2-3-108-18-

Output